SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName:B. Tech		**Assignment Type: Lab**	**AcademicYear:**2025-2026
CourseCoordinatorName	Venkataramana Veeramsetty		
Instructor(s)Name	1. Dr. Mohammed Ali Shaik 2. Dr. T Sampath Kumar 3. Mr. S Naresh Kumar 4. Dr. V. Rajesh 5. Dr. Brij Kishore 6. Dr Pramoda Patro 7. Dr. Venkataramana 8. Dr. Ravi Chander 9. Dr. Jagjeeth Singh		
CourseCode	24CS002PC215	**CourseTitle**	AI Assisted Coding
Year/Sem	II/I	**Regulation**	R24
Date and Day of Assignment		**Time(s)**	
Duration	2 Hours	**Applicableto Batches**	
AssignmentNumber:3.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1	Lab 3: Prompt Engineering – Improving Prompts and Context Management **Lab Objectives:** • To understand how prompt structure and wording influence AI-generated code. • To explore how context (like comments and function names) helps AI generate relevant output. • To evaluate the quality and accuracy of code based on prompt clarity. • To develop effective prompting strategies for AI-assisted programming. **Lab Outcomes (LOs):** After completing this lab, students will be able to: • Generate Python code using Google Gemini in Google Colab. • Analyze the effectiveness of code explanations and suggestions by Gemini. • Set up and use Cursor AI for AI-powered coding assistance. • Evaluate and refactor code using Cursor AI features. • Compare AI tool behavior and code quality across different platforms.	03.08.2025 EOD

Task Description#1

- Try 3 different prompts to generate a factorial function.

Expected Output#1

- Comparison of AI-generated code styles

Prompt 1:Write a python code that take input from the console and a print stmt for calculating the factorial of a number

Code:

```python
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

number = int(input("Enter a number: "))
try:
    fact = factorial(number)
    print(f"The factorial of {number} is {fact}")
except ValueError as e:
    print(e)
```

Output:

```
Problems (Ctrl+Shift+M)
Problems   Output   Debug Console   Terminal   Ports

PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>  & 'c:\Users\anuma\AppData\Local\Microsoft\Window
a\.cursor\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher' '59274
top\AIAC\Lab-3\task1.py'
Enter a number: 5
The factorial of 5 is 120
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
Ctrl+K to generate a command
```

Prompt 2: Write a python code that do not take input from the console to calculate factorial of a number

Code:

```python
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbe
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

number = 6  # You can change this value to calculate factorial of ano
fact = factorial(number)
print(f"The factorial of {number} is {fact}")
```

Output:

```
Problems   Output   Debug Console   Terminal   Ports
ibs\debugpy\launcher' '59848' '--' 'c:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3\task1-2.py'
The factorial of 6 is 720
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Prompt 3: Write a python function that calculate factorial of a number 8

Code:

```
task1-1.py   task1-2.py   task1-3.py ×   Launch
task1-3.py > ...
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative numbers.")
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

number = 8
fact = factorial(number)
print(f"The factorial of {number} is {fact}")
Ctrl+L to chat, Ctrl+K to generate
```

Output:

```
c:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3\task1-3.py
The factorial of 8 is 40320
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Comparision: All three codes calculate factorial using the same logic, but they differ in how the input is given. task1.py takes input from the user at runtime, making it interactive. task1_2.py uses a hardcoded value 10, so it always calculates factorial of 10. task1_3.py also uses a hardcoded value 5, but adds a helpful comment suggesting the value can be changed. In short, the difference lies only in input method—user input vs fixed value.

Task Description#2

- Provide a clear example input-output prompt to generate a sorting function.

Expected Output#2

- Functional sorting code from AI

Prompt 1: write a python code that Provide a clear example input-output prompt to generate a sorting function.

Code:

```python
def sort_numbers(numbers):
    """
    Sorts a list of numbers in ascending order.

    Args:
        numbers (list): A list of numbers to sort.

    Returns:
        list: Sorted list of numbers.
    """
    return sorted(numbers)

# Example input-output prompt:
example_input = [5, 2, 9, 1, 5, 6]
print("Input:", example_input)
print("Output:", sort_numbers(example_input))
```

Output:

```
Problems   Output   Debug Console   Terminal   Ports

.13.exe' 'c:\Users\anuma\.cursor\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\d
neDrive\Desktop\AIAC\Lab-3\task2-1.py'
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Prompt 2: write a python code without the function that generate a sorting function

Code:

```python
# Generate a sorting function (bubble sort) and assign it
sort_numbers = lambda numbers: sorted(numbers)

# Example usage:
example_input = [5, 2, 9, 1, 5, 6]
print("Input:", example_input)
print("Output:", sort_numbers(example_input))
Ctrl+L to chat, Ctrl+K to generate
```

Output:

```
:\Users\anuma\AppData\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\anuma\.cursor\extensions\ms-p
ython.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher' '61409' '--' 'c:\Users\anuma\OneDrive\De
sktop\AIAC\Lab-3\task2-2.py'
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
Ctrl+K to generate a command
```

Prompt 3: write a python code that generate the sorting numbers without using function and lambda

Code:

task2-3.py > ...

```python
numbers = [5, 2, 9, 1, 5, 6]
numbers.sort()
print("Sorted numbers:", numbers)
Ctrl+L to chat, Ctrl+K to generate
```

Output:

```
\launcher  61918  --  C:\Users\anuma\OneDrive\D
Sorted numbers: [1, 2, 5, 5, 6, 9]
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Comparision: All three codes sort a list but differ in approach. task2_1.py and task2_2.py use Python's inbuilt sorted() function, making them short, efficient, and practical for real-world use. task2_3.py implements sorting manually using the Bubble Sort algorithm, which is longer and slower.

Task Description#3

- Start with the vague prompt "Generate python code to calculate power bill" and improve it step-by-step

Expected Output#3

- Enhanced AI output with clearer prompts

Prompt 1: write a python code to calculate power bill

Code:

task3-1.py > ...

```python
def calculate_power_bill(units):
    """
    Calculates the power bill based on the number of units consumed.
    Example slab:
        - First 100 units: $0.5 per unit
        - Next 100 units (101-200): $0.75 per unit
        - Next 100 units (201-300): $1.20 per unit
        - Above 300 units: $1.50 per unit
    """
    bill = 0
    if units <= 100:
        bill = units * 0.5
    elif units <= 200:
        bill = 100 * 0.5 + (units - 100) * 0.75
    elif units <= 300:
        bill = 100 * 0.5 + 100 * 0.75 + (units - 200) * 1.20
    else:
        bill = 100 * 0.5 + 100 * 0.75 + 100 * 1.20 + (units - 300) * 1.50
    return bill
```

```python
# Example usage:
units_consumed = 350  # You can change this value to test with different units
bill_amount = calculate_p (variable) units_consumed: Literal[350]
print(f"Units Consumed: {units_consumed}")
print(f"Power Bill: ${bill_amount:.2f}")
```

Output:

```
Units Consumed: 350
Power Bill: $320.00
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Prompt 2:Write a python code that take units from the console to calculate power bill

Code:

```python
def calculate_power_bill(units):
    """
    Calculates the power bill based on the number of units consumed.
    Example slab:
        - First 100 units: $0.5 per unit
        - Next 100 units (101-200): $0.75 per unit
        - Next 100 units (201-300): $1.20 per unit
        - Above 300 units: $1.50 per unit
    """
    bill = 0
    if units <= 100:
        bill = units * 0.5
    elif units <= 200:
        bill = 100 * 0.5 + (units - 100) * 0.75
    elif units <= 300:
        bill = 100 * 0.5 + 100 * 0.75 + (units - 200) * 1.20
    else:
        bill = 100 * 0.5 + 100 * 0.75 + 100 * 1.20 + (units - 300) * 1.50
    return bill
```

```python
try:
    units_consumed = float(input("Enter the number of units consumed: "))
    if units_consumed < 0:
        print("Units consumed cannot be negative.")
    else:
        bill_amount = calculate_power_bill(units_consumed)
        print(f"Units Consumed: {units_consumed}")
        print(f"Power Bill: ${bill_amount:.2f}")
except ValueError:
    print("Invalid input. Please enter a numeric value for units.")
```

Problems Output Debug Console **Terminal** Ports

```
' 'c:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3\lab3-2.py'
Enter the number of units consumed: 100
Units Consumed: 100.0
Power Bill: $50.00
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Prompt 3: Write a python code that calculate power bill from the different types of

customers

Code:

```python
def calculate_power_bill(customer_type, units_consumed):
    """
    Calculate the power bill based on customer type and units consumed.

    Args:
        customer_type (str): Type of customer - 'residential', 'commercial', or
        units_consumed (float): Number of units consumed

    Returns:
        float: Total bill amount
    """
    customer_type = customer_type.lower()
    if customer_type == 'residential':
        # Example: First 100 units at $0.12/unit, next 200 at $0.15, rest at $0
        if units_consumed <= 100:
            bill = units_consumed * 0.12
        elif units_consumed <= 300:
            bill = 100 * 0.12 + (units_consumed - 100) * 0.15
```

```python
    else:
        bill = 100 * 0.12 + 200 * 0.15 + (units_consumed - 300) * 0.20
elif customer_type == 'commercial':
    # Example: First 200 units at $0.16/unit, rest at $0.22
    if units_consumed <= 200:
        bill = units_consumed * 0.16
    else:
        bill = 200 * 0.16 + (units_consumed - 200) * 0.22
elif customer_type == 'industrial':
    # Example: Flat rate $0.25/unit, minimum bill $500
    bill = units_consumed * 0.25
    if bill < 500:
        bill = 500
else:
    raise ValueError("Invalid customer type. Choose 'residen
return bill
```

```python
# Example usage:
if __name__ == "__main__":
    print("Power Bill Calculator")
    ctype = input("Enter customer type (residential/commercial/industrial): ").
    try:
        units = float(input("Enter units consumed: "))
        total_bill = calculate_power_bill(ctype, units)
        print(f"Total bill for {ctype} customer consuming {units} units: ${tota
    except ValueError as e:
        print("Error:", e)
```

Output:

```
Data\Local\Microsoft\WindowsApps\python3.13.exe' 'c:\Users\anuma\.cursor\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debu
auncher' '64237' '--' 'c:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3\task3-3.py'
Power Bill Calculator
Enter customer type (residential/commercial/industrial): industrial
Enter units consumed: 1000
Total bill for industrial customer consuming 1000.0 units: $500.00
Total bill for industrial customer consuming 1000.0 units: $500.00
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Comparision: These three codes all calculate electricity bills but differ in flexibility. task3_1.py uses fixed rates (Residential = 5, Commercial = 8, Industrial = 10) for billing, making it the simplest. task3_2.py improves flexibility by letting the user enter custom per-unit rates for each type before billing. task3_3.py is the most advanced, as it allows defining multiple slabs (consumption ranges with different rates) for each customer type, making it closer to real-world billing systems. In short, the difference is in rate handling—fixed, user-defined, and slab-based.

Task Description#4

- Write structured comments to help AI generate two linked functions (e.g., login_user() and register_user())..

Expected Output#4

- Consistent functions with shared logic

Prompt1: write a python code that generates to help AI generate two linked functions (e.g e.g., login_user() and register_user()).that take input from the console.

Code:

task4-1.py > ...

```python
def register_user(users):
    username = input("Enter a new username: ")
    if username in users:
        print("Username already exists. Please try a different one.")
        return False
    password = input("Enter a new password: ")
    users[username] = password
    print("Registration successful!")
    return True

def login_user(users):
    username = input("Enter your username: ")
    password = input("Enter your password: ")
    if username in users and users[username] == password:
        print("Login successful!")
        return True
    else:
        print("Invalid username or password.")
        return False
```

```python
# Example usage:
if __name__ == "__main__":
    users = {}
    while True:
        print("\n1. Register\n2. Login\n3. Exit")
        choice = input("Choose an option: ")
        if choice == "1":
            register_user(users)
        elif choice == "2":
            login_user(users)
        elif choice == "3":
            print("Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")
```

Output:

```
1. Register
2. Login
3. Exit
2. Login
3. Exit
Choose an option: 1
Enter a new username: niha
Enter a new password: niha
Registration successful!
```

Promt2: write a python code that generates to help AI generate two linked functions (e.g e.g., login_user() and register_user()).that do not take input from the console.

Code:

```python
def register_user(users, username, password):
    if username in users:
        print(f"Registration failed: Username '{username}' already exists.")
        return False  # Username already exists
    users[username] = password
    print(f"Registration successful for user '{username}'.")
    return True  # Registration successful
                (parameter) users: Any
def login_user(users, username, password):
    if username in users and users[username] == password:
        print(f"Login successful for user '{username}'.")
        return True  # Login successful
    else:
        print(f"Login failed for user '{username}'. Invalid username or password.
        return False  # Invalid username or password
```

```python
# Example usage:
users = {}
# Register a user
register_user(users, "alice", "password123")
# Try to login
login_success = login_user(users, "alice", "password123")
Ctrl+L to chat, Ctrl+K to generate
```

Output:

```
Problems   Output   Debug Console   Terminal   Ports
Login successful for user 'alice'.
Registration successful for user 'alice'.
Login successful for user 'alice'.
Login successful for user 'alice'.
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Conclusion: The codes show two different implementations of a user registration and login system in Python. The key difference lies in their main logic loop and the user's ability to exit.

The first version is designed to force the user to register and log in before they can proceed. The while loop continues as long as logged_in is False. There is no option to exit the program without successfully logging in. The loop terminates only after a successful login, at which point it prints "You are now logged in. Proceed with your tasks."

The second version gives the user more control by providing an exit option. The while True loop runs indefinitely until the user explicitly chooses to exit by selecting option "3". The break statement then terminates the loop and the program. This version doesn't force a login; it allows the user to register, log in, or exit at their discretion.

Task Description#5

- Analyzing Prompt Specificity: Improving Temperature Conversion Function with Clear Instructions

Expected Output#5

- Code quality difference analysis for various prompts

Prompt1:write a python code for temperature conversion from Fahrenheit to Celsius

Code:

```python
def fahrenheit_to_celsius(fahrenheit):
    """
    Convert temperature from Fahrenheit to Celsius.

    Args:
        fahrenheit (float): Temperature in Fahrenheit.

    Returns:
        float: Temperature in Celsius.
    """
    celsius = (fahrenheit - 32) * 5 / 9
    return celsius

# Example usage:
if __name__ == "__main__":
    f = float(input("Enter temperature in Fahrenheit: "))
    c = fahrenheit_to_celsius(f)
    print(f"{f} Fahrenheit is {c:.2f} Celsius.")
```

Output:

```
\libs\debugpy\launcher' '53305' '--' 'c:\Users\anuma\OneDrive\Desktop\AIAC\
Enter temperature in Fahrenheit: 98
98.0 Fahrenheit is 36.67 Celsius.
```

Prompt2: write a python code for temperature conversion from Fahrenheit to Celsius that do not take input from the console

Code:

```python
def fahrenheit_to_celsius(fahrenheit):
    return (fahrenheit - 32) * 5 / 9

# Example usage:
 (variable) celsius_temp: float
celsius_temp = fahrenheit_to_celsius(fahrenheit_temp)
print(f"{fahrenheit_temp}°F is equal to {celsius_temp:.2f}°C")
```

Output:

```
Problems   Output   Debug Console   Terminal   Ports

77°F is equal to 25.00°C
PS C:\Users\anuma\OneDrive\Desktop\AIAC\Lab-3>
```

Conclusion:
The two Python code snippets both deal with temperature conversion, but they differ in their structure and modularity.
The first code uses a single function, temperature_conversion(), to handle both Celsius to Fahrenheit and Fahrenheit to Celsius conversions. It prompts the user for their choice of conversion type within this function and then performs the calculation and prints the result directly.
In contrast, the second code demonstrates a more modular approach. It defines a dedicated

function, fahrenheit_to_celcius(), specifically for converting Fahrenheit to Celsius. This function takes a Fahrenheit temperature as input and returns the calculated Celsius value, separating the conversion logic from the user interaction and output. If a Celsius to Fahrenheit conversion was also needed, a separate function for that would likely be created, promoting code reusability and better organization.

Evaluation Criteria:

Criteria	Max Marks
Factorial Function (Task#1)	0.5
Sorting Function (Task#2)	0.5
Vogue Vs. Specific Prompting (Task #3)	0.5
Linked Functions (Task #4)	0.5
Temperature Conversion Function (Task #5)	0.5
Total	**2.5 Marks**